

```
body { font-family: helvetica; font-size: 10pt; color: #333333; } h1 { font-family: helvetica; font-size: 18pt; font-weight: bold; color: #1a365d; margin-top: 15px; margin-bottom: 10px; } h2 { font-family: helvetica; font-size: 14pt; font-weight: bold; color: #2c5282; margin-top: 12px; margin-bottom: 8px; } h3 { font-family: helvetica; font-size: 11pt; font-weight: bold; color: #4a5568; margin-top: 10px; margin-bottom: 6px; } p { margin-bottom: 8px; line-height: 1.4; } ul { margin-bottom: 8px; } li { margin-bottom: 4px; } table { border-collapse: collapse; width: 100%; margin-top: 10px; margin-bottom: 10px; } th { background-color: #ebf8ff; color: #2b6cb0; font-weight: bold; border: 1px solid #cbd5e0; padding: 6px; } td { border: 1px solid #cbd5e0; padding: 6px; } code { font-family: courier; font-size: 9pt; background-color: #f7fafc; } blockquote { border-left: 3px solid #cbd5e0; padding-left: 10px; color: #4a5568; margin-left: 10px; }
```

INFORME FINAL DE AUDITORÍA DE SISTEMAS

Link: **Repositorio GitHub:** https://github.com/fevervc/AUDITORIA_EXAMEN_3

CARÁTULA

Entidad Auditada: CORPORATE EPIS PILOT - Sistema Mesa de Ayuda con Inteligencia Artificial

Ubicación: Lima, Lima, Perú

Período auditado: 24/06/2026

Equipo Auditor: Estudiante - Curso de Auditoría de Sistemas

Fecha del informe: 24/06/2026

Supervisor: Docente del Curso de Auditoría de TI

ÍNDICE

1. Resumen Ejecutivo
 2. Antecedentes
 3. Objetivos de la Auditoría
 4. Alcance de la Auditoría
 5. Normativa y Criterios de Evaluación
 6. Metodología y Enfoque
 7. Hallazgos y Observaciones
 8. Análisis de Riesgos
 9. Recomendaciones
 10. Conclusiones
 11. Plan de Acción y Seguimiento
 12. Anexos
-

1. RESUMEN EJECUTIVO

Se realizó la auditoría de sistemas del producto **Mesa de Ayuda con Inteligencia Artificial (AI Help Desk)** perteneciente a **CORPORATE EPIS PILOT**. El sistema fue clonado desde el repositorio del curso, desplegado en entorno Docker con el modelo de lenguaje `smollm:360m` de Ollama, y sometido a pruebas funcionales, de seguridad y de código.

El sistema presenta una **arquitectura RAG bien estructurada** con separación de responsabilidades entre frontend (React/TypeScript), backend (FastAPI/LangChain) y proxy (NGINX). Se identificaron **5 hallazgos de auditoría**, dos de criticidad alta, dos de criticidad media y uno de criticidad baja. El flujo completo de conversación, clasificación de intenciones y creación de tickets fue verificado y funciona correctamente tras las correcciones aplicadas.

Estado general del sistema: OPERATIVO CON OBSERVACIONES.

2. ANTECEDENTES

CORPORATE EPIS PILOT es un sistema de asistencia conversacional con IA diseñado para entornos empresariales. Su propósito es responder dudas de usuarios basándose en una base de conocimiento interna (documentos PDF y TXT), guiar al usuario hacia una solución y, si no se resuelve el problema, crear un ticket de soporte técnico.

El sistema fue desarrollado como proyecto de demostración utilizando tecnologías open-source ejecutadas

completamente en local, sin dependencia de APIs de terceros de pago. Forma parte del catálogo de proyectos del Curso de Auditoría de TI de la EPIS.

No se registran auditorías previas del sistema.

3. OBJETIVOS DE LA AUDITORÍA

Objetivo General

Evaluar el funcionamiento, la arquitectura, la seguridad y la calidad del código del sistema de Mesa de Ayuda con IA de CORPORATE EPIS PILOT, con el fin de verificar su conformidad con las mejores prácticas de desarrollo de software y su eficiencia en el soporte a las operaciones de la organización.

Objetivos Específicos

- Evaluar la infraestructura y el despliegue en contenedores:** Verificar la correcta operatividad e integración de los servicios del sistema (Proxy, Frontend, Backend y Ollama) en entorno Docker. (Evidenciado en [ANX-01](#) y [ANX-02](#))
- Validar la precisión de la recuperación y flujo conversacional RAG:** Evaluar el proceso de ingesta de documentos, clasificación de intenciones y la interacción con la base vectorial ChromaDB. (Evidenciado en [ANX-03](#) y [ANX-05](#))
- Verificar el flujo de gestión de incidentes y persistencia:** Confirmar la creación y correcto registro en base de datos de los tickets de soporte técnico generados por los usuarios. (Evidenciado en [ANX-04](#))
- Analizar la calidad del código, arquitectura y seguridad:** Identificar vulnerabilidades de control de acceso, configuración CORS y buenas prácticas en los componentes de software del sistema. (Evidenciado en [ANX-06](#))

4. ALCANCE DE LA AUDITORÍA

- **Ámbitos evaluados:** Tecnológico, funcional, de seguridad y de calidad de código.
- **Sistemas incluidos:**
 - Backend: API FastAPI con LangChain, Ollama (smollm:360m), ChromaDB, SQLite
 - Frontend: Aplicación React/TypeScript/MUI
 - Infraestructura: Docker Compose, NGINX Proxy
- **Áreas auditadas:**
 - Código fuente del backend (main.py, ingest.py, database_setup.py)
 - Código fuente del frontend (App.tsx)
 - Configuración de infraestructura (docker-compose.yml, nginx.conf, Dockerfile)
 - Base de conocimiento (documentos RAG)
 - Base de datos SQLite (tickets.db)
- **Período auditado:** 24 de junio de 2026

5. NORMATIVA Y CRITERIOS DE EVALUACIÓN

No	Norma / Marco	Aplicación
1	ITIL 4 (Foundation)	Gestión de incidentes y tickets de soporte
2	OWASP Application Security v4.0.3	Seguridad de la API REST y manejo de datos
3	ISO/IEC 27001:2022	Seguridad de la información y control de accesos
4	Buenas prácticas de desarrollo Python (PEP 8)	Calidad de código del backend
5	Docker Security Best Practices	Configuración segura de contenedores

6. METODOLOGÍA Y ENFOQUE

La auditoría siguió un **enfoque mixto** (basado en riesgos y cumplimiento), aplicando los siguientes métodos:

- **Inspección de código fuente:** Revisión estática de todos los archivos Python, TypeScript y de configuración.
- **Pruebas funcionales:** Ejecución del sistema en entorno Docker y pruebas directas a la API REST mediante Invoke-RequestMethod de PowerShell.
- **Análisis de logs:** Revisión de los logs estructurados del backend (loguru/JSON) mediante docker compose logs.
- **Pruebas de conectividad:** Verificación de la comunicación entre contenedores y con el servicio Ollama del host.
- **Verificación de base de datos:** Consulta directa a la base SQLite desde el contenedor para confirmar persistencia de tickets.
- **Revisión documental:** Análisis del readme.md del sistema, requirements.txt y manifiestos de infraestructura.

7. HALLAZGOS Y OBSERVACIONES

Hallazgo 1 (H-01): Sin autenticación en endpoints de la API

Descripción: El endpoint /ask de FastAPI no requiere ningún tipo de autenticación. Cualquier cliente con acceso a la red puede consumir el servicio LLM ilimitadamente.

Evidencia objetiva: backend/main.py línea 111: @app.get("/ask") sin dependencia de autenticación.

Grado de criticidad: [ALTO] **ALTO**

Criterio vulnerado: OWASP API Security Top 10 - API1: Broken Object Level Authorization; ISO/IEC 27001 A.9.4.2

Causa: El sistema fue diseñado como prototipo interno sin capa de seguridad.

Efecto: Exposición del servicio a abuso, consumo excesivo de recursos del modelo LLM y fuga potencial de información de la base de conocimiento.

Hallazgo 2 (H-02): Volumen de base de datos montado sin inicialización

Descripción: El docker-compose.yml monta el archivo ./backend/tickets.db del host (0 bytes) sobre /app/tickets.db del contenedor, sobrescribiendo la base de datos inicializada durante el build y causando el error no such table: tickets.

Evidencia objetiva: Log del backend: ERROR | main:ask_question:154 - Error en el endpoint /ask: no such table: tickets. Archivo host tickets.db verificado con tamaño 0 bytes.

Grado de criticidad: [ALTO] **ALTO**

Criterio vulnerado: Buenas prácticas de Docker (gestión de volúmenes); ITIL 4 - Gestión de incidentes.

Causa: Discrepancia entre la inicialización en tiempo de build y el bind mount del host.

Efecto: El sistema no puede crear tickets, bloqueando el flujo principal de soporte.

Hallazgo 3 (H-03): CORS configurado con comodín sin restricción de origen

Descripción: El middleware CORS de FastAPI acepta peticiones de cualquier origen (allow_origins=["*"]).

Evidencia objetiva: backend/main.py línea 47: allow_origins=["*"].

Grado de criticidad: [MEDIO] **MEDIO**

Criterio vulnerado: OWASP ASVS v4.0 - V14.4.1; ISO/IEC 27001 A.13.1

Causa: Configuración por defecto para facilitar el desarrollo.

Efecto: En producción, permite que sitios maliciosos realicen solicitudes a la API en nombre de usuarios autenticados.

Hallazgo 4 (H-04): Base de datos SQLite sin cifrado

Descripción: Los tickets de soporte (que pueden contener información sensible de los usuarios) se almacenan en un archivo SQLite sin ningún cifrado.

Evidencia objetiva: backend/database_setup.py: sqlite3.connect('tickets.db') sin cifrado.

Grado de criticidad: [MEDIO] **MEDIO**

Criterio vulnerado: ISO/IEC 27001 A.10.1 - Controles criptográficos; OWASP ASVS V6.

Causa: Elección de tecnología orientada a simplicidad de prototipo.

Efecto: Si el archivo tickets.db es exfiltrado, todos los datos de soporte quedan expuestos en texto plano.

Hallazgo 5 (H-05): Calidad de respuestas limitada por el modelo smollm:360m

Descripción: El modelo smollm:360m (361M parámetros, Q4_0) produce respuestas de calidad inferior al modelo original (llama3.1:8b), mezclando idiomas y generando contenido impreciso o irrelevante en consultas RAG.

Evidencia objetiva: Respuesta a la pregunta "Que es Anviltron" generó texto en inglés y conceptos incorrectos (plataforma de marketing) en lugar de la descripción del producto real.

Grado de criticidad: [BAJO] **BAJO** (en contexto académico/prueba)
Criterio vulnerado: ITIL 4 - Garantía de calidad del servicio.
Causa: Sustitución del modelo original por uno de menor capacidad para fines de prueba académica.
Efecto: Experiencia de usuario degradada; respuestas incorrectas pueden desorientar a los usuarios reales.

8. ANÁLISIS DE RIESGOS

Hallazgo	Riesgo Asociado	Impacto	Probabilidad	Nivel de Riesgo
H-01	Abuso de la API y DoS al servicio LLM	Alto	Alta	[ALTO] Alto
H-02	Sistema no funcional en despliegue frío	Alto	Alta	[ALTO] Alto
H-03	Ataques CSRF desde sitios maliciosos	Medio	Media	[MEDIO] Medio
H-04	Exposición de datos de soporte	Medio	Baja	[MEDIO] Medio
H-05	Respuestas incorrectas al usuario	Medio	Alta	[MEDIO] Medio

9. RECOMENDACIONES

R-01 (para H-01): Implementar autenticación mediante **API Key** o **JWT Bearer Token** en todos los endpoints de la API. Usar el sistema de dependencias de FastAPI (Depends) para requerir el token en cada solicitud.

R-02 (para H-02): Reemplazar el bind mount de `tickets.db` por un **volumen nombrado de Docker**:

```
volumes:
tickets_data:
services:
backend:
volumes:
- tickets_data:/app/tickets.db
```

Alternativamente, eliminar el volumen y usar `database_setup.py` como entrypoint del contenedor.

R-03 (para H-03): Configurar CORS con orígenes específicos en lugar de comodín:

```
allow_origins=["http://localhost:5173", "https://tudominio.com"]
```

R-04 (para H-04): Migrar la base de datos a **PostgreSQL** o implementar **SQLCipher** para cifrado transparente de SQLite. Considerar usar variables de entorno para credenciales de conexión.

R-05 (para H-05): En entornos de producción, utilizar un modelo LLM de mayor capacidad (ej. `llama3.1:8b`, `mistral:7b`) o integrar una API de LLM cloud con calidad garantizada (OpenAI, Anthropic) con protección de datos adecuada.

10. CONCLUSIONES

El sistema **Mesa de Ayuda con IA de CORPORATE EPIS PILOT** demuestra una arquitectura tecnológica moderna y bien estructurada, implementando correctamente los patrones RAG, router de intenciones y bucle de feedback. El uso de herramientas open-source (FastAPI, LangChain, ChromaDB, Ollama, React) y la containerización con Docker representan buenas prácticas de desarrollo.

Sin embargo, el sistema en su estado actual **no es apto para despliegue en producción** sin antes resolver los hallazgos de criticidad alta (H-01 y H-02), que comprometen tanto la seguridad como la operatividad del sistema.

El flujo funcional fue verificado exitosamente: - [OK] Clasificación de intenciones (saludo, pregunta, problema) - [OK] Recuperación de información desde base vectorial ChromaDB - [OK] Bucle de feedback (¿se resolvió tu problema?) - [OK] Creación y persistencia de tickets en SQLite - [OK] Comunicación interna entre contenedores Docker

Los controles existentes son parcialmente adecuados para un entorno de demostración, pero requieren mejoras significativas para cumplir con los estándares ITIL 4, OWASP y ISO/IEC 27001 en un entorno empresarial real.

11. PLAN DE ACCIÓN Y SEGUIMIENTO

Hallazgo	Recomendación	Responsable	Fecha Comprometida
H-01	Implementar autenticación JWT/API Key	Equipo de Desarrollo	30/07/2026
H-02	Migrar a volumen nombrado Docker	DevOps / Infraestructura	15/07/2026
H-03	Restringir orígenes CORS a dominio propio	Equipo de Desarrollo	15/07/2026
H-04	Migrar a PostgreSQL o cifrar SQLite	Equipo de Desarrollo	31/08/2026
H-05	Evaluar modelo LLM de mayor capacidad	Arquitectura de IA	31/08/2026

12. ANEXOS

Los siguientes documentos de evidencia se encuentran en la carpeta [/evidencias/](#) del repositorio:

Anexo	Archivo	Contenido
ANX-01	ANX-01-servicios-docker.md	Servicios Docker activos (docker ps)
ANX-02	ANX-02-modelo-smollm.md	Verificación modelo smollm:360m en Ollama
ANX-03	ANX-03-ingesta-rag-vectorstore.md	Ingesta de documentos y base vectorial
ANX-04	ANX-04-creacion-tickets-sqlite.md	Creación de ticket y verificación DB
ANX-05	ANX-05-pruebas-conversacionales.md	Resultados de pruebas funcionales
ANX-06	ANX-06-revision-codigo-fuente.md	Revisión de código fuente y arquitectura

Lima, 24 de junio de 2026

Estudiante Auditor
Curso de Auditoría de Sistemas de TI
Escuela Profesional de Ingeniería de Sistemas (EPIS)

Supervisor
Docente del Curso de Auditoría de TI
EPIS - Universidad

Este informe fue elaborado como parte del Examen de Auditoría de Sistemas. Los hallazgos y recomendaciones representan el resultado de la evaluación técnica del sistema CORPORATE EPIS PILOT en la fecha indicada.